

Relatório Técnico - GeoLog

Plataforma de Telemetria Logística e Persistência Poliglota (SQLite + MongoDB)

Instituição: UNIPÊ | **Disciplina:** Tópicos Avançados em Banco de Dados / Arquitetura de Software | **Professor:** Me. Ricardo Roberto de Lima

Equipe: _____ (preencher com os nomes dos integrantes)

1. Visão geral

A LogiTech Express monitora uma frota de caminhões que envia coordenadas GPS, temperatura e velocidade com alta frequência, além de manter cadastros de motoristas, veículos e ordens de serviço. Esses dois tipos de dado têm naturezas diferentes, então a solução adota **persistência poliglota**: cada dado fica no banco mais adequado, e a aplicação Streamlit (arquivo único `geolog_app.py`) unifica tudo em uma interface com mapa interativo e dashboard.

2. Arquitetura poliglota

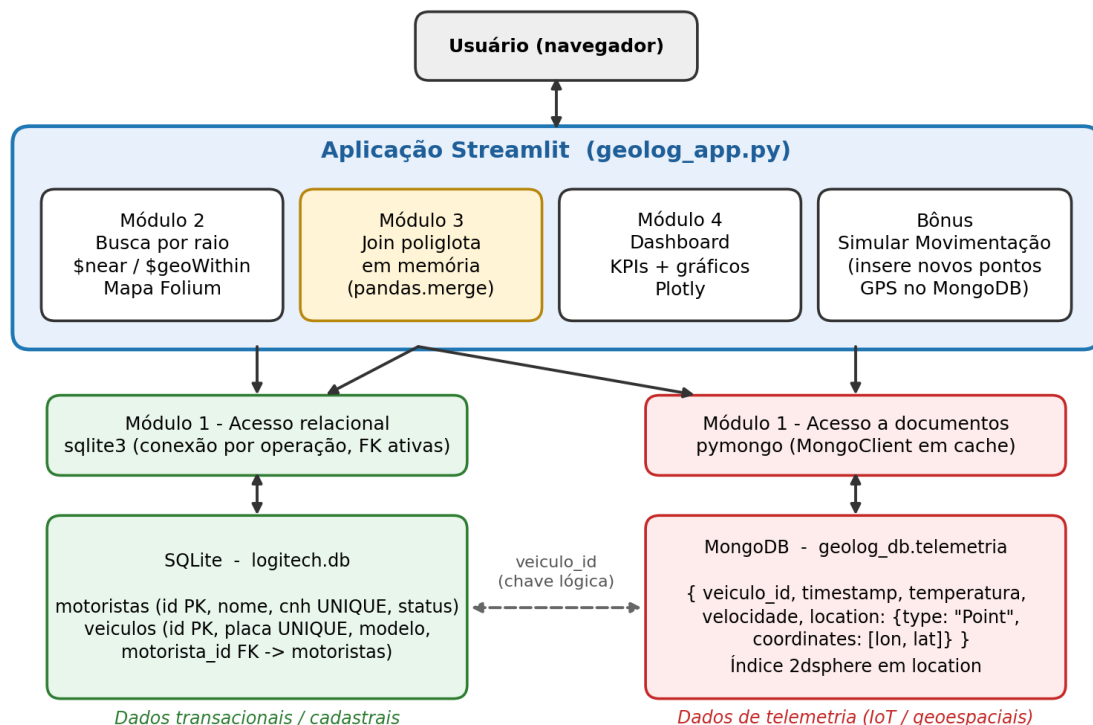


Figura 1 - Camadas da aplicação. O vínculo entre os bancos é lógico (`veiculo_id`): não há chave estrangeira entre SQLite e MongoDB, e o cruzamento é feito na aplicação.

3. Por que dois bancos?

Dado	Banco	Justificativa
Motoristas e veículos (cadastro)	SQLite	Estrutura fixa, integridade referencial (FK de veículo para motorista), UNIQUE em CNH e placa, CHECK no status e consultas com JOIN.
Telemetria (GPS, temperatura, velocidade)	MongoDB	Documentos flexíveis e de alta frequência, formato GeoJSON e suporte nativo a índice 2dsphere e busca por proximidade.

4. Módulos implementados

Módulo 1 - Persistência e seed. Na inicialização, o app cria as tabelas `motoristas` e `veiculos` no `logitech.db` (com `INSERT OR IGNORE`, idempotente) e, no MongoDB, cria o índice **2dsphere** em `location` por código e carrega a coleção `geolog_db.telemetry` se estiver vazia. Além dos 3 registros do enunciado, o seed inclui 6 leituras anteriores por veículo (histórico sintético e determinístico) para que o gráfico de temperatura e a rota no mapa tenham mais de um ponto.

Módulo 2 - Busca por raio. O usuário escolhe o ponto de referência, o raio (1 a 50 km) e o operador (`$near` ou `$geoWithin`). A consulta considera apenas a última posição de cada veículo. O mapa Folium mostra o

círculo do raio, o ponto de referência, marcadores dos veículos dentro do raio (vermelho se houver alerta de velocidade), a rota recente e os veículos fora do raio em cinza.

```
db.telemetria.find({_id: {$in: ultimos_ids}, location: {$near: {$geometry: {type: 'Point', coordinates: [lon, lat]}, $maxDistance: raio_km * 1000}}})

db.telemetria.find(..., location: {$geowithin: {$centerSphere: [[lon, lat], raio_km / 6378.1]]}})
```

Módulo 3 - Join poliglota em memória. O cadastro vem do SQLite (JOIN entre veículos e motoristas) e a última leitura de cada veículo vem de um *aggregate* no MongoDB (\$sort + \$group com \$first). Os dois resultados são cruzados com pandas.merge pela chave veiculo_id, gerando a tabela: Motorista | Placa | Última Temperatura | Velocidade | Coordenadas.

Módulo 4 - Dashboard. KPIs de frotas ativas (motorista com status Ativo), temperatura média da carga e alertas de velocidade (> 80 km/h), além de dois gráficos Plotly: histórico de temperatura por veículo (linha) e distribuição do status dos motoristas (pizza).

Bônus - Simulador. O botão *Simular Movimentação* insere no MongoDB um novo ponto por veículo, com pequena variação aleatória de coordenadas, temperatura e velocidade e timestamp sempre posterior ao anterior. Como o Streamlit reexecuta o script a cada clique e os botões são tratados antes das abas, mapa, tabela e dashboard já saem atualizados, sem reiniciar a aplicação.

5. Como executar

```
pip install streamlit pymongo folium streamlit-folium plotly pandas
docker run -d --name mongo -p 27017:27017 mongo:7
streamlit run geolog_app.py
```

O endereço do MongoDB pode ser alterado pela variável de ambiente MONGO_URI (padrão mongodb://localhost:27017). Se o MongoDB estiver fora do ar, o app mostra uma mensagem de erro clara em vez de quebrar.

6. Decisões e limitações

- Uma conexão SQLite por operação, porque o sqlite3 não é compartilhável entre threads e o Streamlit roda cada interação em uma thread diferente; o MongoClient é thread-safe e fica em cache.
- Timestamps ficam como texto ISO 8601 em UTC (igual ao seed do enunciado), que ordena corretamente de forma lexicográfica.
- Não há integridade referencial entre os bancos: telemetria de um veiculo_id sem cadastro no SQLite fica fora da visão unificada (o merge parte do cadastro), então a consistência entre os dois lados é responsabilidade da aplicação.
- O simulador cresce a coleção a cada clique; o botão *Resetar telemetria* restaura o seed inicial.